

AhamX Bodhi

Where I Becomes Infinite

WordPiece Tokenization

Module 3.3: Tokenization

Prof. Sashikumaar Ganesan | IISc Bengaluru



Session Overview

What You Will Learn

- WordPiece's likelihood-based merge criterion
- How WordPiece training differs from BPE
- The ## prefix convention for subwords
- WordPiece encoding at inference time
- BERT, DistilBERT, and Electra applications

Concept at a Glance

- **Duration:** 8 minutes
- **Bloom's Level:** B3 (Apply)
- **Difficulty:** L2 (Intermediate)
- **Key Idea:** Maximize language model likelihood
- **Used By:** BERT, DistilBERT, Electra, MobileBERT



Learning Objectives

By the End of This Concept You Will Be Able To

- **Explain** how WordPiece selects merges using a likelihood scoring criterion
- **Contrast** WordPiece with BPE in terms of training objective and vocabulary quality
- **Interpret** the ## prefix convention and how it marks subword continuations
- **Trace** WordPiece encoding on a short example string
- **Identify** which BERT-family models rely on WordPiece and why





Historical Context

From Google's Neural Machine Translation Work

- WordPiece was developed at Google in 2012 (Schuster and Nakamura) for Japanese/Korean speech recognition
- Later described in the Google Neural Machine Translation system (Wu et al., 2016)
- Popularized in the NLP community through **BERT** (Devlin et al., 2018)
- Motivation: BPE's raw frequency criterion does not explicitly optimize language model quality

The key innovation: instead of merging the most *frequent* pair, WordPiece merges the pair that most **improves the likelihood** of the training data under a unigram language model.



The Likelihood Merge Criterion

How WordPiece Scores Each Merge

- For each candidate pair (A, B) , compute the **likelihood score**:

$$\text{score}(A, B) = \frac{f(AB)}{f(A) \cdot f(B)}$$

- $f(X)$ = frequency of symbol X in the current corpus
- This is a **pointwise mutual information** (PMI)-like measure
- High score means A and B co-occur far more often than chance
- Merge the pair with the **highest** score at each step



BPE vs WordPiece: The Merge Decision

BPE: Raw Frequency

- Merge criterion: $\max f(AB)$
- Favours very common bigrams regardless of whether they form a meaningful unit
- Simple and fast to compute
- May merge frequently co-occurring but linguistically arbitrary pairs

WordPiece: Likelihood Gain

- Merge criterion: $\max \frac{f(AB)}{f(A) \cdot f(B)}$
- Favours pairs whose co-occurrence is *surprising* given individual frequencies
- Tends to produce more linguistically coherent subword units
- Slightly more expensive to compute





WordPiece Training

Training Steps

- **Step 1:** Initialize vocabulary with all unique characters in the training corpus
- **Step 2:** Mark continuation characters with ## prefix: ``play'' $\rightarrow$ p, ##l, ##a, ##y
- **Step 3:** Compute likelihood score for every adjacent symbol pair
- **Step 4:** Merge the highest-scoring pair, update the corpus, re-score
- **Step 5:** Repeat until vocabulary size V is reached (BERT uses 30,522 tokens)
- Unlike BPE, WordPiece does **not** store merge rules — it stores the final vocabulary only



The ## Prefix Convention

Marking Subword Continuations

- In WordPiece, every subword token that does **not** start a word carries a ## prefix
- This signals: "I am a continuation piece, not a word start"
- Example: ``unaffable'' → [un, ##aff, ##able]
- Example: ``tokenization'' → [token, ##ization]
- Reconstructing the original word: remove ## and concatenate
- BPE instead attaches whitespace prefix `` ' ' to word-starting tokens



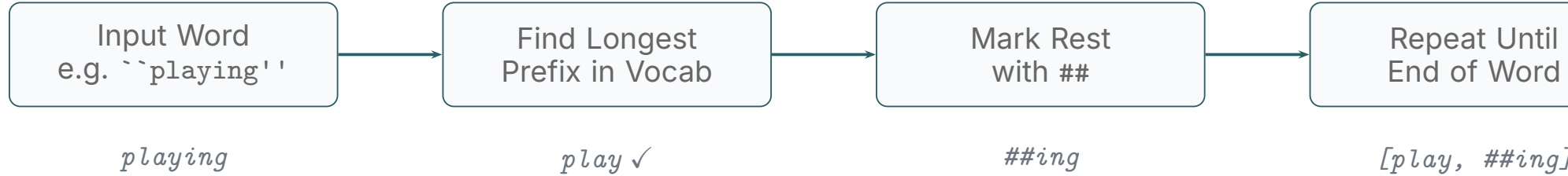
WordPiece Encoding at Inference

Greedy Longest-Match-First Algorithm

- WordPiece does **not** replay merge rules like BPE; instead it uses a vocabulary lookup strategy
- For each word, find the longest prefix that exists in the vocabulary
- Mark the remainder with ## and repeat from the split point
- Example: ``playing'' with vocab containing play and ##ing: → [play, ##ing]
- If no prefix exists, fall back to character-level splits: p, ##l, ##a, ...



WordPiece Encoding: Step-by-Step



Unlike BPE which replays merge rules sequentially, WordPiece inference is a vocabulary lookup problem solvable with a simple greedy scan from left to right.

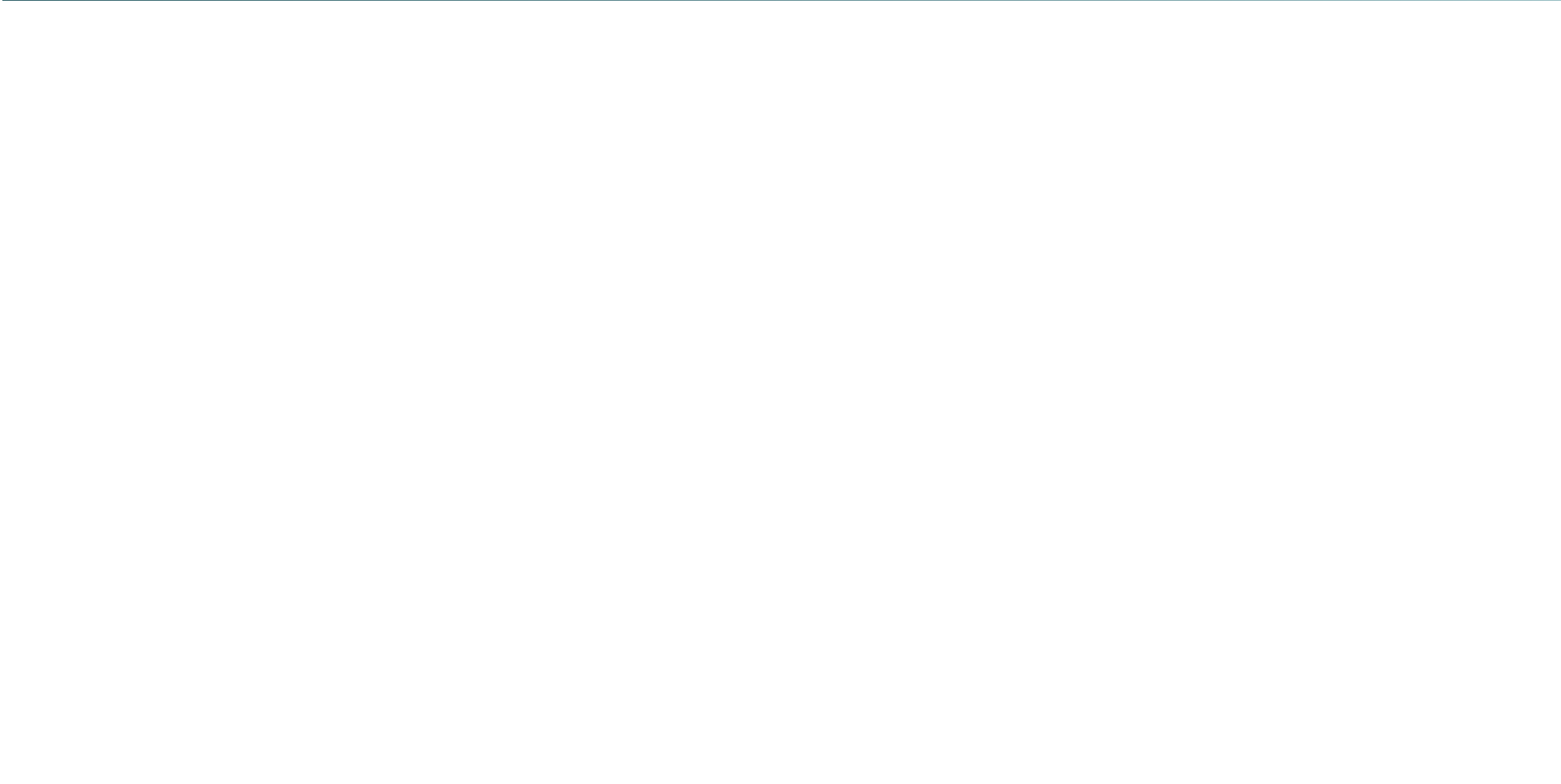


BERT's WordPiece Vocabulary

BERT Tokenizer Details

- BERT-base uses a vocabulary of **30,522 tokens** for English
- BERT-multilingual-base covers 104 languages in **119,547 tokens**
- Special tokens: [CLS] (classification), [SEP] (separator), [MASK], [PAD], [UNK]
- The vocabulary was built from English Wikipedia and the BookCorpus (~3.3 billion words)

The [CLS] token is a critical design choice in BERT: its final hidden state is used as the **sentence-level representation** for classification tasks, making the special token meaningful beyond just a sequence delimiter.





GenAI Application: BERT Embeddings in Production

BERT as a Text Embedding Engine

- BERT's WordPiece tokenizer is used in thousands of production embedding services
- **Sentence-BERT (SBERT):** Encodes sentences into 768-dimensional vectors using BERT's WordPiece
- **Google Search:** BERT-based models power contextual understanding of search queries
- **Pinecone / Weaviate:** Vector databases ingest BERT-tokenized encodings for semantic search

Every time a RAG pipeline uses a BERT-based retriever to find relevant document chunks, WordPiece tokenization is the first step — the query and document are both tokenized before embedding.



GenAI Application: Question Answering with BERT

WordPiece in Extractive QA

- Extractive QA (e.g., SQuAD) requires the model to identify answer *spans* within a passage
- The [SEP] token separates the question from the context passage in the input
- The model outputs start and end *token* indices — subword aware, not character aware
- Post-processing reconstructs the answer string by removing ## and joining tokens
- HuggingFace Transformers pipeline handles this WordPiece decoding automatically



GenAI Application: Fine-Tuned Text Classifiers

BERT in Production Classifiers

- Sentiment analysis (DistilBERT)
- Intent classification in chatbots
- Spam and content moderation filters
- Named entity recognition (NER)

WordPiece Impact

- Subword tokenization handles product names, domain jargon, and typos
- The classifier never sees [UNK] for technical terms
- Consistent tokenization ensures reproducible predictions



GenAI Application: WordPiece in the HuggingFace Ecosystem

HuggingFace Tokenizers Library

- **AutoTokenizer.from_pretrained("bert-base-uncased"):** Loads the BERT WordPiece tokenizer in one line
- **tokenizers.WordPieceTrainer:** Train custom WordPiece vocabularies from domain-specific corpora
- **Fast tokenizers:** Rust-backed implementations run at ~1 million tokens/second
- **Offset mapping:** WordPiece tokenizer returns character-level offsets for NER post-processing
- Over 100,000 models on HuggingFace Hub use a WordPiece-based tokenizer



WordPiece vs BPE: Summary Table

Property	WordPiece	BPE
Merge Criterion	Likelihood score (PMI-like)	Raw frequency
Stores at inference	Vocabulary only	Vocabulary + merge rules
Subword marker	##(prefix on continuation)	Whitespace prefix on word-start
Encoding algorithm	Greedy longest-match-first	Replay merge rules in order
Key Models	BERT, DistilBERT, Electra	GPT-2/4, Llama 3, Falcon
Vocabulary Size	30K (BERT-base)	50K–128K



Common Misconceptions About WordPiece

Myths and Corrections

- **Myth:** "WordPiece is just BPE with different branding." They differ fundamentally in training objective and inference algorithm
- **Myth:** "## means the token is less important." The prefix is purely a reconstruction marker with no semantic weight
- **Myth:** "WordPiece handles unknown characters with [UNK]." Characters not in vocabulary cause [UNK]; the fix is byte-level extension
- **Myth:** "WordPiece is used in GPT models." GPT models use BPE; WordPiece is the BERT family standard

Key Takeaways



What You Have Learned

- WordPiece selects merges by maximizing a **likelihood score** proportional to PMI, not raw frequency
- The ## prefix marks continuation subword pieces, enabling lossless word reconstruction
- Inference uses a greedy longest-match-first vocabulary scan, not merge rule replay
- BERT, DistilBERT, and Electra all use WordPiece with a 30K vocabulary
- WordPiece powers the BERT embedding ecosystem: semantic search, QA, classification, and NER in production
- HuggingFace Tokenizers provides fast, production-grade WordPiece training and inference

Thank You

WordPiece Tokenization — Module 3.3